# API Documentation Generator

## API Notes

ILLM Project 31

Detailed Project Report

Department of Artificial Intelligence

Sardar Vallabhbhai National Institute of Technology Surat

B Tech Artificial Intelligence 2023 to 2027

**Submitted by**

Vishal Gautam U23AI127

Divyansh Kumar U23AI082

Tanu Meena U23AI076

Hanmant Jajulwar U23AI098

November 2026

Source analysis, usage example retrieval and local Gemma descriptions

# Abstract and contents

API Notes is a local API documentation generator that reads Python API source and supplied HTTP usage examples. It extracts route and schema facts with Python abstract syntax trees, retrieves compatible examples with a custom sparse TF-IDF implementation, and asks the pretrained Gemma 2B model to write endpoint explanations through Ollama. Markdown and OpenAPI 3.1 exports retain the extracted contract and source references.

The implementation separates declared API facts from generated prose. On ten manually annotated routes, the AST workflow recovered all ten method and path pairs, while a local-decorator regex baseline recovered one. The selected unigram retrieval setting achieved 83.33% hit@1 and 100% hit@3 on twelve held-out authored queries. An expanded study captured sixteen actual model generations across eight endpoints in three demonstration APIs. Both prompt settings averaged 1.50 out of 2 for correctness in a source-based assistant review. Usage context corrected one return-value error, but the study did not show a consistent quality improvement.

The result is a working prototype for common FastAPI and Flask declarations. Source evidence and review remain necessary because valid citation identifiers do not establish factual accuracy. This report explains the design, algorithms, implementation, evaluation, testing, limitations and reproduction steps in greater detail than the accompanying presentation.

## Contents


| Section | Page |
|---|---|
| Problem and background | 3 |
| Scope and requirements | 4 |
| System architecture | 5 |
| Static source analysis | 6 |
| Usage example retrieval | 7 |
| Local language model generation | 8 |
| Implementation and exports | 9 |
| Worked Bookstore example | 10 |
| Application workflow and error cases | 11 |
| Evaluation design | 12 |
| Route and retrieval results | 13 |
| Language model results | 14 |
| Source based case analysis | 15 |
| Verification and operation | 16 |
| Limitations and future work | 17 |
| References and reproduction | 18 |

# Problem and background

## Documentation from changing source code

API consumers need to know which URL to call, which HTTP method to use, what input to send and what response to expect. These details often appear in several places: a route decorator declares the local path, an application file mounts a router, a model describes fields, and a handler contains error branches. A short docstring rarely explains all of them. Manually assembling a reference creates an opportunity for omissions whenever the implementation changes.

The project topic is API Documentation Generator. Its objective is to generate documentation directly from source code and usage examples with an LLM. API Notes addresses that topic through a Python-first workflow that reconstructs supported declarations, attaches relevant example requests, drafts descriptions and makes the result available for review and export.

## Why source analysis and language generation are combined

An abstract syntax tree, or AST, represents the structure of a program without running it. This makes it useful for extracting explicit route decorators, type annotations, defaults and model declarations. The tree does not automatically resolve all runtime behavior. Dynamic registration, inherited validation and middleware logic can remain outside the supported analysis.

A language model can turn visible code into a readable explanation, but it can also introduce a claim that the code does not support. The implementation therefore stores the declared contract independently and gives the model a restricted task: write prose about supplied evidence. Reviewers can compare a description with its handler and model snippets instead of relying on the generated wording alone.

## Usage examples as additional context

A supplied HTTP example shows a concrete request method, URL and optional body. It can help identify the intended use of an endpoint and clarify field values. API Notes ranks these examples with TF-IDF and cosine similarity, then filters them by method and route compatibility. This is retrieval over local authored examples rather than an external knowledge base. Examples are parsed as documentation inputs and are not executed.

The evaluation separately measures route extraction, example retrieval and generated prose. That separation matters because a correct method and path do not imply a correct explanation, and a relevant request example does not prove the behavior of the response. The recorded results support specific findings on the demonstration set, rather than a general accuracy claim. [1, 2, 3]

# Scope and requirements

## Supported inputs and outputs

The application accepts up to thirty source and example files with a combined limit of 500,000 UTF-8 bytes. At least one file must be Python source. Accepted example extensions are .http, .rest and .txt. Each uploaded name is a relative label. The validation rejects absolute paths, parent-directory traversal, unsupported extensions and duplicate names. Supplying the related modules helps resolve imports and router mounting.

| Area | Current support |
| --- | --- |
| Framework declarations | Literal FastAPI and APIRouter routes, Flask and Blueprint routes |
| Routing | Common imports and aliases, static mounts, prefixes, Flask prefix overrides |
| Parameters | Typed path, query and body inputs, defaults, simple Annotated declarations |
| Models | Direct BaseModel fields, required values, defaults and common constraints |
| Responses | Declared response models, HTTPException or abort codes, simple Flask status tuples |
| Exports | Markdown and OpenAPI 3.1 JSON with source attribution |

## Functional requirements

The workflow must produce a usable reference before an LLM description is available. Each endpoint should show its method and path, declared inputs, response information and source evidence. Compatible HTTP examples should remain accessible. The user can draft one endpoint description, save records, inspect changes between revisions and download both export formats.

## Operational requirements

The generator runs as a local standard-library HTTP service with a browser interface. Fresh model descriptions require a reachable Ollama service and an installed model. The readiness indicator distinguishes unavailable Ollama, a missing model, active generation and readiness. An unavailable model prevents a new generation job from starting, while source analysis and existing exports remain usable.

This prototype focuses on common Python declarations. It does not fully reconstruct dynamic routes, custom decorator wrappers, inherited Pydantic behavior or middleware authentication. Untyped bodies and responses can remain unknown. JavaScript and Express are future extensions. These are limits of the current implementation. [1, 4, 5]

# System architecture

API Notes processes source files and HTTP examples through two related paths. The parser constructs a declared reference from AST nodes. The example index builds sparse vectors and ranks requests. The engine combines an endpoint with compatible requests, then sends its contract and evidence to the local model when the user requests a description.

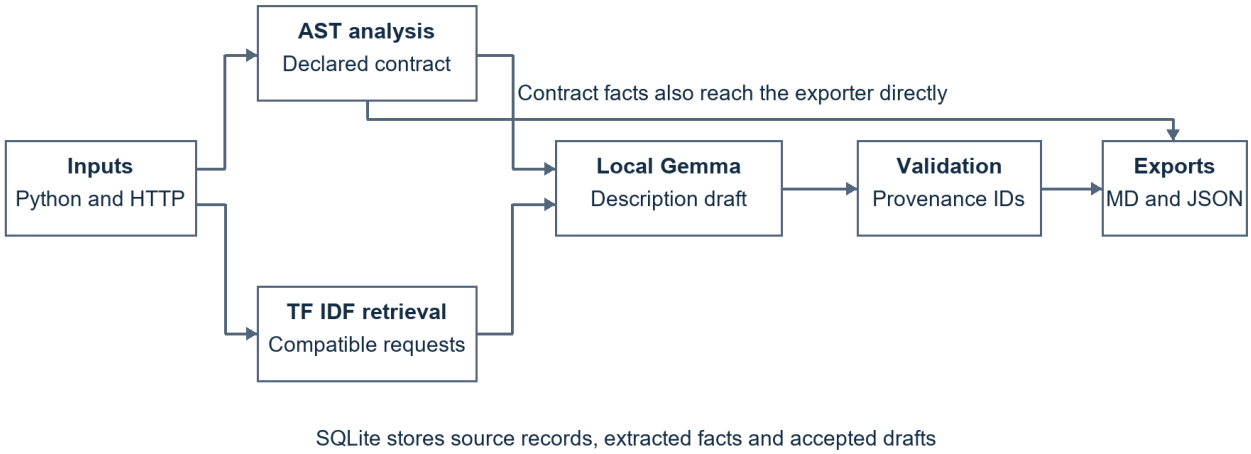



*Figure 1 Documentation generation and the direct path for extracted facts*

## Data flow and separation of responsibilities

The parser returns endpoint records, named schemas, source snippets and analysis notes. Each evidence snippet includes a relative file name, line range and stable source identifier. The retrieval component returns ranked example records with their identifiers and similarity scores. The engine applies route compatibility and keeps the endpoint contract available independently of prose.

The model receives the declared method, path, parameters and responses alongside handler and model evidence. Its output passes a JSON and provenance validator before the server attaches it to the saved endpoint. The exporter takes structural facts from the parsed record and text from the accepted draft where available. A generated description therefore cannot replace the method, path, type or status code in the contract.

## Persistence and the browser interface

SQLite stores project records as JSON bodies with a project identifier and creation time. Transactional updates preserve existing fields while a generation thread records progress or attaches a draft. The browser shows extracted schema tables, source evidence, usage examples, saved records and evaluation cases. The interface retains the source-derived reference when generation fails.

The main distinction is between extracted declarations, supplied examples and generated explanations. They have different evidence strengths. The architecture preserves their origins so that a reviewer can assess the description against the available code. [4, 5, 6, 7]

# Static source analysis

## AST parsing and module resolution

app/parser.py calls ast.parse on each supplied Python file and records syntax errors as analysis notes. It never imports or executes the submitted application. For a valid module, it collects imports, recognized application and router instances, and directly declared BaseModel classes. Import mappings let it relate an alias in one module to a router instance defined in another.

A second pass identifies include_router and register_blueprint calls. The parser propagates static mount information through recognized instances until there are no new mounts, with a bounded number of passes. It accounts for FastAPI prefix composition and the Flask behavior in which an explicit registration url_prefix replaces the Blueprint prefix. Missing or dynamic relationships produce notes instead of a claim that a runtime route has been fully resolved.

## A prefixed route example

```
router = APIRouter(prefix="/books")
@router.post("", response_model=Book, status_code=201)
def create_book(book: BookCreate):
    ...

app.include_router(router, prefix="/v1")
```

The endpoint path combines the application mount /v1, the router prefix /books and the decorator path, which is empty in this example. The resulting route is POST /v1/books. A regex that reads only the local decorator finds the empty path and cannot reconstruct the mount. This is the concrete issue tested by the extraction comparison.

## Parameters models and responses

The parser reads function annotations, defaults and recognized Query, Path, Body and Field declarations. Model fields carry JSON-compatible types, required membership, defaults and supported constraints such as minimum or maximum length and numeric bounds. It links declared response models to schemas and finds explicit HTTPException or abort calls within handlers. A simple Flask return tuple can declare a success status.

Evidence identifiers use a truncated SHA-256 digest over the file label, start line and snippet content. These identifiers link the handler and model definitions to the endpoint and support provenance validation. They do not verify that a model has interpreted the code correctly. Complex runtime behavior and unsupported schema constructs still need review. [4, 5, 10]

# Usage example retrieval

## Text representation

app/retrieval.py implements tokenization, sparse term vectors and cosine ranking with the Python standard library. Tokenization splits camelCase, replaces underscores with spaces, lowercases text and retains alphanumeric tokens. The compared feature modes use words alone, words plus adjacent bigrams, or those features plus character n-grams of lengths two, three and four within padded words.

The example index fits inverse document frequencies on the example texts. It uses logarithmic term frequency and smoothed inverse document frequency, then normalizes each vector to unit L2 length. Query features absent from the fitted vocabulary are ignored. A zero vector produces no positive-scoring results. The following equations match the implementation, with N example documents and document frequency df for term t.

$$\mathrm{tf}_{t,d} = 1 + \ln(c_{t,d})$$

$$\mathrm{idf}_t = \ln\left(\frac{N + 1}{\mathrm{df}(t) + 1}\right) + 1$$

$$w_{t,d} = \mathrm{tf}_{t,d} \times \mathrm{idf}_t \qquad \text{Unit vectors use} \qquad \frac{w}{\|w\|_2}$$

$$\cos(q, d) = \hat{q} \cdot \hat{d}$$

Here c is the count of a term within a text, w is its weighted vector, and the normalized vectors are denoted by a hat. Because both vectors have unit length, cosine similarity is their dot product. The implementation iterates over the smaller sparse dictionary for the product.

## Ranking and route compatibility

The engine builds a query from the endpoint method, path, function name, docstring and declared parameter names. It asks the index to rank the examples, then retains requests whose HTTP method matches and whose URL path fits the endpoint template. A placeholder such as {book_id} matches one non-empty path segment. The engine stores at most three compatible examples, while the model prompt includes at most two.

This compatibility step has a different purpose from semantic ranking. It prevents a highly similar request for another route from entering an endpoint description. Query parameters do not change the URL path used for compatibility, but they remain visible in the example text.

## Configuration selection

The evaluation compares all feature modes on the same ten-example index. It uses four development queries to choose the runtime setting, followed by twelve held-out paraphrases for reporting. All three settings tie on the development metrics. The current max operation retains the first tied setting, unigram. The character setting performs better on the recorded test queries, but those test results do not determine the runtime configuration. [2, 4, 6]

# Local language model generation

## Model and prompt construction

API Notes uses the existing pretrained gemma:2b model through Ollama. The team does not train or fine-tune the model. app/llm.py posts a non-streaming chat request to the local /api/chat endpoint. The default service address is http://127.0.0.1:11434. The configured model is replaceable through OLLAMA_MODEL.

The prompt includes the method and path, JSON parameter and response records, up to four source snippets and up to two compatible request examples. Each source snippet is capped at 5,000 characters and each request at 1,800 characters. The source-only setting uses the same contract and source evidence with no request examples. Generation uses temperature 0, a 4,096-token context and up to 650 predicted output tokens. The request timeout is 150 seconds.

The instructions ask for a short summary and a paragraph about visible behavior. They tell the model to treat source and request text as data and to avoid inventing authentication, persistence, response fields or timing guarantees. The output schema constrains field names and allowed citation and parameter identifiers. A concrete limitations list is required.

| Output field | Purpose |
|---|---|
| summary | Short endpoint description |
| behavior | Explanation of visible handler behavior |
| parameter_notes | Notes keyed by declared parameter names |
| evidence_ids | Identifiers for supplied source snippets |
| example_ids | Identifiers for supplied request examples |
| limitations | Limitations of the explanation or visible evidence |

## Validation and remaining factual risk

The validator rejects extra top-level fields, empty or oversized prose, unavailable source and example identifiers, undeclared parameter notes and invalid limitations lists. It removes duplicate citation identifiers and attaches model, timing and mode metadata after validation. It also rejects invalid JSON and output-limit failures.

These checks establish format and identifier consistency. They do not prove semantic accuracy. A draft can cite a real handler while describing its return value incorrectly, or mention an undeclared parameter in unrestricted prose. The expanded Bookstore case demonstrates both. Review must therefore inspect all description fields, including the limitations text, against the source. [3, 7, 8]

# Implementation and exports

## Module organization

| Module | Responsibility |
|---|---|
| app/parser.py | AST parsing, mounts, schemas, endpoint facts and source evidence |
| app/retrieval.py | HTTP request parsing, feature counts, TF-IDF and cosine ranking |
| app/engine.py | Input validation, compatible context, exports and revision comparison |
| app/llm.py | Readiness, prompt creation, Ollama request and output validation |
| app/quality.py | Review hashes, score validation and comparison summaries |
| app/storage.py | SQLite records and transactional updates |
| app/server.py | Local HTTP endpoints and asynchronous generation |
| static/ | Browser pages, schema tables, readiness and comparison rendering |

## Storage and generation jobs

SQLite keeps each project in a projects table with an integer key and a serialized JSON body. The application exposes identifiers such as DOC-0001. Write-ahead logging and a 15-second connection timeout support local access, while BEGIN IMMEDIATE protects updates. A single generation lock prevents concurrent model jobs. A background thread records the current endpoint, completed drafts, processed endpoints and errors without discarding the source record.

## Markdown and OpenAPI

The Markdown exporter includes method and path headings, summary and behavior text, input tables, response statuses, example requests, source line references and schema JSON. The OpenAPI exporter writes version 3.1.0, paths, operations, parameters, request bodies, responses and reusable schemas. Operation x-source extensions retain handler attribution, and x-analysis-notes retains parser warnings.

Both exports use extracted structural facts. The model may supply the summary, behavior and parameter descriptions after validation. The OpenAPI file is a static draft rather than the framework-generated runtime schema. Form and file inputs, embedded bodies, aliases and duplicate model names can require corrections before production use.

## Revision comparison

The engine compares endpoint keys and serialized parameter and response contracts between records. It reports added, removed and changed routes and whether schemas changed. This supports reviewing declared API changes, but it does not establish a full behavioral or semantic code diff. [4, 5, 7, 9]

# Worked Bookstore example

## Input and declared contract

The Bookstore sample consists of app.py, routes.py, models.py and requests.http. The application mounts the book router under /v1. The create route has an empty decorator suffix under the /books router prefix, so the complete endpoint is POST /v1/books. Its signature declares a BookCreate body, a Book response model and status 201.

| Field | Type | Required | Declaration |
|---|---|---|---|
| title | string | Yes | Length 1 to 120 |
| author | string | Yes | No additional constraint declared |
| price | number | Yes | Minimum 0 |
| published | boolean | No | Default false |

The response Book model adds an integer id to the title, author, price and published fields. These are separately declared response fields. The sample source does not declare external database persistence or authentication.

## Visible implementation

```
book_id = max(BOOKS, default=0) + 1
saved = {"id": book_id, **book.model_dump()}
BOOKS[book_id] = saved
return saved
```

The handler selects one more than the maximum existing ID, or one for an empty catalog. It creates a dictionary from the ID and model fields, stores that dictionary in the in-memory BOOKS catalog and returns the complete saved record. That last statement is central to reviewing the generated explanation.

## Supplied request

```
POST http://localhost:8001/v1/books
Content-Type: application/json

{"title":"The Pragmatic Programmer",
 "author":"Andrew Hunt", "price":499, "published":true}
```

The engine associates EX-books-create with the POST endpoint after ranking and compatibility filtering. The request values demonstrate one intended input. They do not prove runtime validation or a response body because the generator does not execute this request. The source evidence and the declared response schema provide the basis for describing the return behavior.

A complete review should check the path, body fields, status 201, next-ID calculation, in-memory storage and full returned dictionary. The accompanying slides focus on this endpoint because it connects extraction, retrieval, generation and export in one example. [4, 7, 10]

# Application workflow and error cases

## A user session

1. Start the local application and confirm the Gemma readiness state.
2. Load the Bookstore example or supply related source and request files.
3. Generate the API reference and select POST /v1/books.
4. Inspect request and response fields, then open source evidence and the matching request.
5. Draft the endpoint description and compare its statements with the handler.
6. Download Markdown and OpenAPI, then inspect Evaluation or Saved projects.

The interface shows structural information before a model draft is available. Model status refreshes every five seconds, and Recheck model provides an explicit refresh. Generation shows progress and records errors. A missing or unavailable model produces an actionable message without removing the parsed reference. A saved Bookstore draft can illustrate a recorded generation, provided it is presented as earlier output.

## Bookstore missing record

```
if book_id not in BOOKS:
    raise HTTPException(status_code=404, detail="Book not found")
```

GET /v1/books/{book_id} contains an explicit 404 branch. The parser can document the declared error code. A description that only discusses a successful lookup remains incomplete if the predefined review criteria include this branch. The sample is still a source fixture, so its presence does not substitute for an executed API behavior test.

## Inventory duplicate SKU

```
if item.sku in ITEMS:
    raise HTTPException(status_code=409, detail="SKU already exists")
```

The Inventory sample adds a second concrete error condition. POST /api/items declares status 201 on success and raises 409 for a duplicate SKU. ItemCreate declares a non-empty SKU of at most forty characters, a non-negative integer quantity and the default warehouse Surat. The lookup route reports 404 for an unknown SKU. Inventory appears in the expanded LLM comparison but is outside the original ten-route extraction and retrieval benchmark.

## Flask coverage

Support Tickets uses a Blueprint mounted under /api/tickets. Its examples include listing, creating, looking up and closing tickets. The create handler reads JSON dynamically, so the generator cannot reconstruct a complete typed request schema from a signature alone. The explicit return tuple declares success status 201. This illustrates both multi-framework support and a visible boundary of static typing information. [1, 4, 10, 11]

# Evaluation design

## Three separate questions

| Experiment | Inputs | Measured outcome |
|---|---|---|
| Route extraction | 10 annotated routes from Bookstore and Support | Exact method and complete path recovery |
| Example retrieval | 10 requests, 4 development queries, 12 test queries | Hit@1, hit@3 and MRR@3 |
| LLM descriptions | 8 endpoints from 3 APIs, 2 prompt settings | Acceptance and source-based prose review |

evaluation/gold.json contains manually annotated expected routes and retrieval labels. These are authored demonstration fixtures rather than a public benchmark. The route baseline reads literal local decorators with regex and performs basic Flask placeholder conversion. It intentionally omits mount resolution. The AST workflow runs on the same source files, and each prediction is compared with the expected project, method and path tuple.

## Metric definitions

Route precision is the number of correct recovered pairs divided by all predicted pairs. Recall divides correct pairs by expected pairs. F1 is their harmonic mean. Exact comparison means a correct method with an incomplete mounted path still counts as a mismatch. Missing and unexpected tuples remain in the artifact for inspection.

Hit@1 counts the queries whose expected example ranks first. Hit@3 counts those with the expected example among the top three. MRR@3 averages the reciprocal rank, using zero when the expected example is absent from the top three. The retrieval experiment measures query ranking without the engine compatibility filter. Consequently its score describes the example index rather than every step of the final endpoint-context workflow.

## LLM comparison and review integrity

evaluation/llm_cases.json defines the endpoint cases, essential facts and a zero-to-two rubric for correctness, completeness and clarity. scripts/compare_llm.py captures one actual Gemma output for each setting and endpoint. Both settings share the contract and source evidence. Only the second setting adds compatible usage examples. The suite retains inputs, full outputs, timing and rejected attempts.

evaluation/llm_review.json contains an explicit Codex assistant review of all description fields. SHA-256 hashes bind reviews to the captured inputs and outputs. app/quality.py rejects incomplete or mismatched reviews, and the interface shows pending review when captures change. The study uses one generation per setting and endpoint, with no independent human rating or repeated-run variability estimate. [2, 3, 8, 12]

# Route and retrieval results

## Extraction results

| Method | Correct | Predicted | Precision | Recall | F1 |
|---|---|---|---|---|---|
| AST and mount resolution | 10 | 10 | 1.00 | 1.00 | 1.00 |
| Local decorator regex | 1 | 10 | 0.10 | 0.10 | 0.10 |

Both approaches predicted ten route pairs, but the local regex recovered only the unprefixed health route correctly. Its remaining paths lacked application and router or Blueprint mounting information. The AST workflow matched all ten manually annotated pairs. The comparison demonstrates the value of static mount resolution against this simple baseline on these fixtures. It does not compare the parser with a production framework schema generator or establish broad coverage.

## Retrieval results

| Features | Count | Dev hit@1 | Test hit@1 | Hit@3 | MRR@3 |
|---|---|---|---|---|---|
| Unigrams | 72 | 75.00% | 83.33% | 100% | 0.8889 |
| Words and bigrams | 182 | 75.00% | 83.33% | 100% | 0.8889 |
| Words, bigrams and characters | 931 | 75.00% | 100.00% | 100% | 1.0000 |

All modes achieved 75% development hit@1, 100% hit@3 and 0.8750 MRR@3. The tie retains unigram because it appears first in the implementation. On the twelve test queries, unigram and word-bigram each put the expected example first for ten queries. Both put it within the top three for every query. The two misses ranked the expected example third, yielding MRR@3 of 0.8889.

The word-character setting recovered all twelve expected examples at rank one, while expanding the vocabulary to 931 features from 72 in the unigram mode. This is a reported test result. Replacing the selected setting using that outcome would use held-out data for selection and require a new independent test set.

## Observed misses and interpretation

The selected mode missed the first rank for the paraphrases about returning only five books and marking a support ticket as closed. Their relevant examples still appeared in the top three. The route compatibility filter can restrict endpoint context after ranking, but this benchmark does not measure an overall documentation-quality benefit from that filter.

The sample size is small and the queries were authored for the fixtures. The percentages should therefore be presented with their counts. Results show behavior on the recorded set and support reproducible implementation comparisons. They do not establish performance on independently selected APIs or natural developer queries. [2, 6, 12]

# Language model results

## Expanded comparison

The expanded suite contains sixteen accepted generations: one source-only draft and one source-plus-usage draft for each of eight endpoints across Bookstore, Support Tickets and Inventory. Both settings use gemma:2b and the same generation options. Acceptance indicates that JSON and provenance checks passed. It is distinct from the source-based review of correctness or completeness.

| Setting | Accepted | Correctness | Completeness | Clarity | Claims |
|---|---|---|---|---|---|
| Source only | 8 of 8 | 1.50 | 1.00 | 1.88 | 4 |
| Source plus usage | 8 of 8 | 1.50 | 1.00 | 1.50 | 4 |

Correctness, completeness and clarity are averages on a zero-to-two rubric. Claims counts the concrete unsupported statements flagged in the assistant review, rather than the number of failed outputs. Scores average accepted descriptions. A rejected attempt has no prose score and must remain visible in the report rather than being silently removed.

## Rubric

Correctness receives zero for wrong core behavior, one for a correct core with a factual error or misleading claim, and two when the statements agree with the supplied source. Completeness receives zero when core behavior is absent, one when the core is covered but an essential fact is missing, and two when all predefined essential facts are covered. Clarity distinguishes unusable, confusing and clear concise prose. Extracted metadata does not count toward generated-prose completeness.

## Interpretation

Usage context corrected the Bookstore create-book return-value statement. Across all eight cases, both settings still introduced unsupported claims or omitted important behavior. Examples include describing an in-memory dictionary as a database or list and leaving out error branches or declared statuses. Both settings average the same correctness and completeness scores, while the source-plus-usage setting has a lower recorded clarity average.

The evidence does not establish a consistent benefit from usage context. It also does not show that context generally harms quality, since the sample is small, the review is assistant-based and there is one output per setting and endpoint. A broader conclusion would require independently chosen projects, blinded human review and repeated captures.

This distinction should remain visible in the presentation: sixteen successful format checks do not mean sixteen factually complete descriptions. The application preserves source context so users can inspect the errors behind the averages. [3, 8, 12]

# Source based case analysis

## Bookstore create endpoint

The expanded study provides a controlled pair for POST /v1/books. The contract declares a BookCreate input, a Book output and status 201. The handler constructs saved as an ID plus all model fields, writes that dictionary to BOOKS and returns saved. The essential facts include the next-ID rule, in-memory storage, full returned record and success status.

```
book_id = max(BOOKS, default=0) + 1
saved = {"id": book_id, **book.model_dump()}
BOOKS[book_id] = saved
return saved
```

## Recorded source only output

The behavior field states: The `create_book` function takes a `BookCreate` object as input and returns a saved book's ID.

This statement changes the returned object into only an ID. The limitations field additionally claims that a required parameter ensures the book is provided, although the signature has no parameter with that name. Both claims appear in an accepted output because the validator checks parameter_notes identifiers and citation availability rather than every statement in prose.

## Recorded source plus usage output

The behavior field states: The `create_book` function takes a `BookCreate` object as input and returns a saved book object.

This version describes the full saved object correctly. It still omits the next-ID calculation and status 201 in its generated description. The limitations field repeats a title-length constraint instead of identifying an unavailable implementation detail or an unverified runtime behavior. That issue illustrates why a structurally valid limitations list can remain unhelpful.

| Setting | Correctness | Completeness | Clarity | Unsupported claims |
|---|---|---|---|---|
| Source only | 0 | 1 | 2 | 2 |
| Source plus usage | 2 | 1 | 2 | 0 |

These are the recorded assistant scores for this endpoint, on the zero-to-two scales. This case supports a specific return-value correction in the captured pair. The aggregate study remains the appropriate place to assess whether that improvement repeats across endpoints. The two kinds of evidence should not be conflated.

A reviewer can use the same procedure for other endpoints: read the contract, inspect the exact source, compare every generated field with the essential facts, list unsupported claims and record the scores against the unchanged output hash. [3, 7, 8, 10]

# Verification and operation

## Verification on 6 November 2026

The existing automated checks were rerun while preparing these deliverables. All thirty Python tests passed, and the frontend smoke script passed. These checks cover routing, source validation, schema declarations, retrieval calculations, model-output validation, persistence, exports, revision comparison, readiness and integrity of the recorded reviews. The source-based quality report was also recomputed in memory and its hashes matched the recorded capture.

| Check group | What the checks establish |
| --- | --- |
| Parser and schemas | Static mounts, aliases, constraints, typed inputs, explicit errors and non-execution |
| Retrieval and metrics | Sparse TF-IDF, cosine behavior and independently calculated metrics |
| Exports and storage | Body schemas, source attribution, revision changes and preserved updates |
| HTTP and readiness | Local assets, request validation, unavailable or busy model handling |
| Review integrity | Changed captures cannot reuse old scores and every case needs review |
| Frontend smoke | Page rendering logic, tabs, schema arrays, readiness transitions and escaping |

Frontend smoke checks are programmatic rendering checks rather than a visual browser walkthrough. Earlier repository documentation records a successful live HTTP rehearsal with actual Gemma generation and both exports. The current verification run covered the Python tests and frontend smoke checks. The recorded browser walkthrough remained pending. Neither workflow tests nor format validation prove prose factuality.

## Run environment

The generator requires Python 3.13 or newer and uses the standard library for runtime operation. FastAPI, Flask and Pydantic appear in analyzed sample source and do not need to be installed to run the generator. Ollama and an installed gemma:2b model are required for new language-model descriptions. Source extraction remains available without them.

```
py -3.13 run_demo.py
# Use another local port if needed
py -3.13 run_demo.py --port 8010
ollama list
```

The launcher opens http://127.0.0.1:8000 by default. OLLAMA_MODEL selects the model and OLLAMA_URL selects the local model service. API_DOC_DB changes the SQLite file location. The server binds to loopback and rejects cross-origin POST requests. These are properties of a local demonstration application, rather than a deployment authentication system. [1, 7, 9, 13]

# Limitations and future work

## Source analysis boundaries

Static analysis covers common literal FastAPI and Flask declarations. Dynamic route construction, conditional mounts, custom decorator wrappers and incomplete imports can prevent full route reconstruction. Direct BaseModel fields work for the supported subset, while inherited validators, custom serialization and duplicate model names require review. Middleware-level authentication and runtime response transformations are outside the complete analysis.

Untyped response bodies and dynamically read request bodies can remain unknown. The exporter is a static OpenAPI draft. Complex form or file inputs, Body embed behavior, aliases and conditional registration can differ from the framework runtime schema. Source references and analysis notes support checking these cases before the output is used beyond the prototype.

## Language model and evaluation boundaries

A constrained JSON response does not prevent factual errors inside valid text. Retrieval supplies authored requests rather than verified executions. The evaluation uses small demonstration APIs, authored queries and an assistant review. It includes one generation per condition and no independent human ratings or uncertainty estimate. These limits prevent broad claims about documentation accuracy, developer time savings or a consistent improvement from usage examples.

## Next experiments and implementation work

A stronger study should use additional independently selected Python projects and manually annotated contracts. At least two reviewers should score randomized, unlabeled output pairs using the same predefined rubric, with disagreements resolved and agreement reported. Repeated captures would estimate variation between outputs. Rejected generations and all full outputs should remain in the published evidence.

Further implementation work can compare the static export with each framework runtime schema in a controlled environment, improve schema alias and inheritance handling, and cover dynamic constructs only where the evidence supports them. A held-out evaluation set is needed before changing retrieval based on the recorded character-feature advantage. Multi-language parsing and response-example support are later extensions.

## Conclusion

API Notes provides a working local workflow for extracting declared Python API contracts, retrieving compatible usage examples and drafting explanations with pretrained Gemma. The team contribution includes the analysis workflow, numerical retrieval implementation, validation, source attribution, exports, persistence, comparison and interface. The recorded extraction and retrieval results are strong on the small fixture set. The LLM study shows why generated prose should remain a reviewable draft with its evidence available. [1, 2, 3, 4]

# References and reproduction

Repository reference: https://github.com/vkg6835/api-documentation-generator. The following numbered sources identify the exact project files used for implementation descriptions, measurements and recorded outputs. They are local project evidence rather than claims of an external research benchmark.

[1] README.md and docs/Presentation_Demo.md. Project purpose, runtime, demonstration and recorded checks.

[2] evaluation/gold.json and artifacts/evaluation.json. Authored route labels, query splits, predictions and metrics.

[3] docs/Evaluation_Method.md and artifacts/llm_quality.json. Expanded study method and reviewed summary.

[4] app/engine.py. Input limits, endpoint queries, compatibility, exports and revision comparison.

[5] app/parser.py. AST workflow, router resolution, schema extraction and evidence identifiers.

[6] app/retrieval.py. Tokenization, TF-IDF implementation and example ranking.

[7] app/llm.py. Model options, prompt, response schema, readiness and provenance checks.

[8] evaluation/llm_cases.json, evaluation/llm_review.json and artifacts/llm_suite.json. Rubric, actual captures and assistant judgments.

[9] app/storage.py and app/server.py. Persistence, HTTP workflow and generation jobs.

[10] examples/bookstore/. Source handlers, models, mounts and supplied requests.

[11] examples/support/ and examples/inventory/. Flask behavior and Inventory errors.

[12] scripts/evaluate.py, scripts/compare_llm.py and app/quality.py. Metrics, captures, review matching and reporting.

[13] tests/ and scripts/rehearse_demo.py. Automated checks and separate live HTTP rehearsal.

## Reproduction commands

```
py -3.13 -m unittest discover -s tests -v
node tests/frontend_smoke.cjs
py -3.13 scripts/evaluate.py
py -3.13 scripts/compare_llm.py --report
```

To capture a new expanded model comparison, use scripts/compare_llm.py --generate with Ollama running. Review every new output and update the matching review before producing scores. Regeneration invalidates old judgments. The optional scripts/evaluate.py --llm captures the original single-endpoint comparison, which is distinct from the expanded eight-endpoint suite.

The submission contains the presentation and this report. Submit one set per team after the final evaluation, before the assignment deadline of 19 November at 23:59.